



ภาพรวมการทำงานของหุ่นยนต์เก็บขยะชายหาด

เอกสารนี้อธิบาย ข้อมูลไหลจากต้นทางไปปลายทางอย่างไร ตั้งแต่การวางแผนเส้นทาง
→ สิ่ง Nav2 → mixer แปลงเป็นความเร็วล้อ → bridge ส่งให้ ESP32 → ESP32 คมมอเตอร์จริง
และข้อมูลที่ ESP32 ส่งกลับขึ้นมา พร้อมทฤษฎีพื้นฐานที่เกี่ยวข้องในแต่ละชั้น

หุ่นยนต์เป็น **skid-steer 4 ล้อแบบไม่สมมาตร** (ล้อหน้าดินตะขาบแคบ 0.734 m, ล้อหลังกลมกว้าง 1.179 m, ระยะ wheelbase 0.950 m) วิ่งบน **ROS2** บน Jetson Nano Super โดยมี ESP32 เป็นตัวคุมมอเตอร์ระดับล่าง

วิธีอ่านเอกสารนี้

ทุกหัวข้อจะมีป้ายบอกว่า "จำเป็นแค่ไหน" เพื่อให้เห็นภาพว่าจะไรคือ **แกนหลักที่ทำงานทุกครั้ง** และอะไรคือ **ของเสริมที่เปิด/ปิดได้**:

ป้าย	ความหมาย
● ใช้หลัก	ทำงาน ทุกครั้ง ที่หุ่นวิ่ง ถ้าขาดส่วนนี้หุ่นไปต่อไม่ได้
○ ทางเลือก	เปิด/ปิดได้ผ่าน flag ตอน launch

อ้างอิงคำสั่งทดสอบบนกระเบื้องภายในอาคาร (tile) ที่ใช้เป็นหลักในเอกสารนี้:
use_gnss:=false (ปิด GNSS), use_zed:=false (ปิด ZED), use_keepout:=false (ปิดแนวรั้ว keepout)
) / coverage_path_mode:=teardrop deadhead_style:=rounded enable_turn_yaw_cut:=true

สายหลัก (core) ที่ต้องมีเสมอ — ถ้าจะจำสั้นๆ คือ 6 ตัวนี้ต่อกันเป็นวง:



ของเสริม (optional) ที่ในคำสั่งนี้ไม่ได้ใช้/ใช้แค่ช่วยด้านความปลอดภัย:

แผนภาพรวม (data flow)



หัวใจคือ วงรอบปิด (closed loop) 2 ชั้น — ลองนึกถึง "คนขับรถ" 2 คนทำงานคนละระดับ:

- **ชั้นบน (ROS2/Nav2):** เหมือนคนขับที่มองถนน — ดูตำแหน่งจาก EKF เทียบกับเส้นทางที่ต้องไป → ตัดสินใจว่าจะ "เดินหน้าเร็วแค่ไหน เลี้ยวแค่ไหน" (`/cmd_vel`)
- **ชั้นล่าง (ESP32):** เหมือนกล่อมเนื้อหา — รับคำสั่งความเร็วล้อ แล้วเทียบกับ encoder

ปรับแรงเหวี่ยง (PWM) ด้วย PID ให้ล้อหมุนได้เร็วตามสั่งจริงๆ แม้พื้นจะลื่นหรือชัน

ทำไมต้อง 2 ชั้น? เพราะชั้นบนคิดช้า (วางแผนทั้งเส้น) ชั้นล่างคิดเร็ว (แก้ทีละล้อหลายร้อยครั้ง/วิ)
แยกกันทำให้แต่ละชั้นโฟกัสงานตัวเองได้เต็มที่

1) การสร้างเส้นทาง (Coverage Planner) 🌀 ใช้หลัก

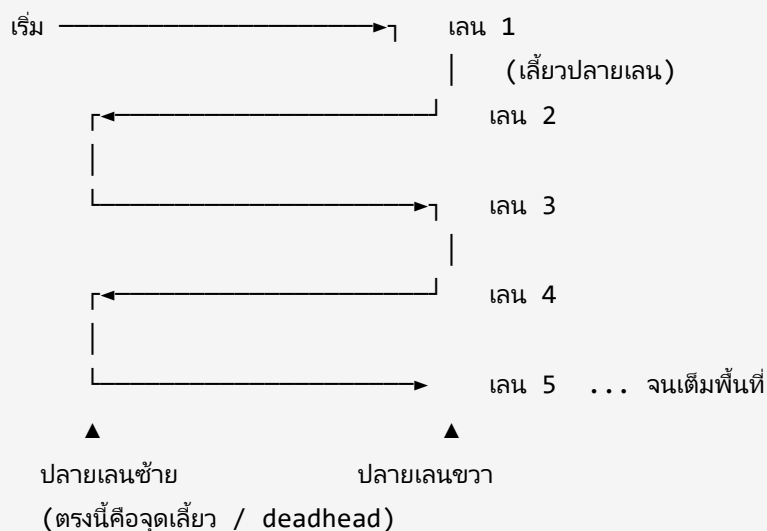
ไฟล์: `beach_robot_coverage_nav2/coverage_follow_waypoints.py`

ทฤษฎี: Boustrophedon (เดินกลับไปกลับมาแบบไถนา)

แบ่งพื้นที่สี่เหลี่ยม (กว้าง × ยาว) ออกเป็น "เลน" ขนานกัน แล้วเดินไป-กลับสลับทิศ

เหมือน การตัดหญ้าสนาม / ไถนา เพื่อให้ scoop (ที่ตัดหญ้า กว้าง 0.6 m) กวาดผ่านครบทั้งพื้นที่
ไม่มีหลุด

ภาพ Boustrophedon (วิ่งไป-กลับสลับทิศ):



ขั้นตอนการสร้าง:

- รับพารามิเตอร์พื้นที่: `area.origin_x/y`, `width`, `height`, `yaw` (กรอบสี่เหลี่ยมที่เรากำหนด)
- หาตำแหน่ง `y` ของแต่ละเลน (`_even_lane_values`) — แบ่งความสูงพื้นที่ออกเป็นช่องเท่าๆ กัน:

```

span = y_บน - y_ล่าง
n     = จำนวนช่อง = ceil(span / tool_width)    ← เลือกให้ระยะเลน ≤ tool_width เสมอ
s     = span / n                               ← ระยะจริงระหว่างเลน (เท่ากันหมด)
y_i   = y_ล่าง + i*s                           (i = 0..n)    ← ตำแหน่ง y ของเลนที่ i

```

เลือก n แบบนี้เพื่อให้ "แถบของ scoop" (กว้าง tool_width) ของเลนติดกันชนกันพอดี → ไม่มีช่องว่าง

3. num_passes (การซ้อนเลนให้ทับ): วางเลนละเอียดที่ระยะ lane_spacing / num_passes

แล้ว interleave (สลับ) เป็น num_passes รอบ — โค้ดคือ fine[p::num_passes] (รอบ p หยิบเลนที่ p, p+N, p+2N...). เลนในรอบเดียวกันจึงห่าง lane_spacing (พอจะเลี้ยว arc ได้) ส่วนรอบถัดไปมาเติมช่องว่างระหว่างเลนเดิม → ชดเชย error การวิ่งเบี้ยว

(ครอบคลุมเต็ม 100% เมื่อ lane_spacing/num_passes ≤ tool_width ; บนทราย)

lane_spacing 3.6, tool 0.6 ต้องใช้ 6 passes ≈ num_passes/6 × 100%, ส่วนคำสั่งกระเบื้อง

lane_spacing 1.2, tool 0.6 ใช้ 3 passes ก็ได้ระยะเลน 0.4 m < 0.6 = ทับซ้อนเต็มพื้นที่)

4. สร้างจุด waypoint ตามแนวเลน (_sample_line) — แบ่งเส้นตรงจากปลายหนึ่งไปอีกปลายเป็นจุดถี่ๆ ด้วยการ แทรกเชิงเส้น (linear interpolation):

```

L = ระยะเลน, n = max(1, LL / waypoint_step)    (waypoint_step = 0.30 m)
จุดที่ k: x_k = x0 + (x1-x0)·(k/n), y_k = y0 + (y1-y0)·(k/n)    (k = 0..n)

```

นี่คือ "ที่มาของตำแหน่งจุด" — ทุกจุดคือจุดบนเส้นตรงระหว่างปลายเลน เว้นระยะเท่าๆ กัน ~0.30 m

5. การเลี้ยวปลายเลน (in-pass arc, _turn_arc): ภายใน pass เดียวกันใช้ครึ่งวงกลม รัศมี = |Δy|/2

(Δy = ระยะระหว่างเลนคู่ที่ต่อกัน) — ครึ่งวงกลมนี้คือ ส่วนโค้งเดียวที่ลากเชื่อมปลายเลนทั้งสอง

ได้พอดีโดยไม่ต้องหยุดหมุนอยู่กับที่ จุดบนโค้งมาจาก $x = x_{end} + r \cdot \cos\theta$, $y = c_y + r \cdot \sin\theta$

6. การย้ายระหว่าง pass (deadhead): เลนคนละรอบ "ไม่ติดกัน" (ห่างเป็นช่วงใหญ่) ครึ่งวงกลมเล็กๆ

จึงเชื่อมไม่ได้ ต้องใช้เส้นทางย้ายตำแหน่งที่ยาวกว่า (รูปแบบขึ้นกับ coverage_path_mode +

deadhead_style — ดูหัวข้อย่อย 1.1)

7. แปลงพิกัด area-local → map (_to_map) — จุดที่คิดในกรอบพื้นที่ ถูกหมุนตาม area.yaw

แล้วเลื่อนไปตาม origin: $x_{map} = origin_x + (lx \cdot \cos \alpha - ly \cdot \sin \alpha)$,

$y_{map} = origin_y + (lx \cdot \sin \alpha + ly \cdot \cos \alpha)$

(คำสั่งนี้ area_yaw:=0 → ไม่หมุน, origin:=0 → ไม่เลื่อน ดังนั้นพิกัด area = map ตรงๆ)

ผลลัพธ์เผยแพร่เป็น /coverage/path (พรีวิวใน RViz) และมี

/coverage/area_boundary (กรอบสี่เหลี่ยมสีฟ้า) แสดงขอบเขตพื้นที่

ทำไมต้องเป็น "เลนไถนาสลับทิศ"? เพราะต้องการให้ scoop กวาดผ่านพื้นที่ครบทุกตารางเมตร

โดย เลี้ยวน้อยที่สุด — เดินเส้นตรงยาวๆ ขนานกันแล้วเลี้ยวกลับหัวที่ปลาย ดีกว่าวิ่งสะเปะสะปะ

(เลี้ยวแต่ละครั้งเสียเวลา + สิ้น + พลาดง่าย) เป็นรูปแบบมาตรฐานของงาน "ปูพื้นที่" (coverage) เช่น หุ่นตัดหญ้า/ดูดฝุ่น

ทำไมต้องส่งทีละช่วง (ไม่ส่งทั้งเส้นรวดเดียว): node ส่ง ทีละ **segment**
(เลน → โค้ง → deadhead) ผ่าน action `FollowPath` เพื่อบังคับให้หุ่นเดินตามเส้นเป๊ะ
(ถ้าใช้ `NavigateThroughPoses` ตัว NavFn planner จะ "ปรับ" เส้นเองทำให้หลุดเลน)

1.1 รูปแบบเส้นทางของคำสั่งบนกระเบื้องภายในอาคาร (tile: teardrop + rounded + yaw-cut) ● ใช้หลัก (ในคำสั่งนี้)

คำสั่งทดสอบบนกระเบื้องใช้

```
coverage_path_mode:=teardrop deadhead_style:=rounded enable_turn_yaw_cut:=true num_passes:=3 turn_r
```

ซึ่ง

ทำให้พฤติกรรมต่างจากค่าเริ่มต้น (`same_direction_loops`) ดังนี้:

ตารางจับคู่: พฤติกรรม ↔ พารามิเตอร์ที่คุมมัน ↔ ค่าที่ใช้ในคำสั่งนี้

พฤติกรรม	คุมด้วย param	ค่าในคำสั่งนี้	ใช้ไหม
สแกนเลนซ้อน (interleave)	<code>num_passes</code>	3	✓ ใช้
เลี้ยวในรอบ = ครึ่งวงกลม (teardrop in-pass)	<code>coverage_path_mode</code>	teardrop	✓ ใช้
รัศมีโค้ง/deadhead	<code>turn_radius</code>	0.60 m	✓ ใช้
ระยะเลนในรอบ	<code>lane_spacing</code>	1.20 m	✓ ใช้
deadhead อยู่ในกรอบ (rounded)	<code>deadhead_style</code>	rounded	✓ ใช้
ตัดโค้งทั้งเมื่อหันหันแล้ว (yaw-cut)	<code>enable_turn_yaw_cut</code>	true	✓ ใช้
gentle merge (เฉียงเข้าเลน)	ไม่มี param — ทำอัตโนมัติเสมอ	—	✓ ทำเสมอ
deadhead อ้อมนอก	<code>deadhead_style</code>	outside	✗ ไม่ใช่ (ต้อง

พฤติกรรม	คุมด้วย param	ค่าในคำสั่งนี้	ใช้ไหม
กรอบ			ปิด keepout)
deadhead หมุนอยู่กับที่	deadhead_style	direct	✗ ไม่ใช้
ทุก transition อ้อมนอก	coverage_path_mode	same_direction_loops (ค่า default)	✗ ถูกแทนด้วย teardrop
หมุนตั้งหัวก่อนเข้าเลน (pre-align Spin)	prealign_at_lane_entry	false (default)	✗ ปิดไว้ (หมุนแล้วศูนย์กลางเลื่อน)

อ่านง่ายๆ: "teardrop" คุม การเลี้ยวในรอบเดียวกัน (ครึ่งวงกลม), "rounded" คุม การย้ายข้ามรอบ (deadhead), ส่วน **gentle merge** ไม่ใช่พารามิเตอร์ — เป็นพฤติกรรมที่โค้ดทำให้เองทุกครั้งที่เริ่มเลนใหม่

รายละเอียดแต่ละพฤติกรรม:

- **num_passes (interleave):** วางเลนละเอียดที่ระยะ `lane_spacing/num_passes` (ในโหมด teardrop ใช้ `_even_lane_values` ซึ่ง "หนึบ" ระยะไม่ให้เกิน `tool_width` → แถบของ scoop ชนกันเสมอ ครอบคลุม 100%) แล้วสานเลนเป็น `num_passes` รอบ; เลนในรอบเดียวกันห่าง `lane_spacing` (เลี้ยว arc ได้), ส่วน offset เต็มช่องว่างระหว่างรอบ
- **เลี้ยวในรอบ (teardrop in-pass)** ← มาจาก `coverage_path_mode:=teardrop` : เมื่อเปลี่ยนเลนภายในรอบเดียวกัน ใช้ครึ่งวงกลม (D-curve, รัศมี `lane_spacing/2`) ที่ปลายเลนฝั่งเดิม หมุนกลับหัวมาเข้าเลนถัดไป (ถ้าไม่ใช่ teardrop จะกลายเป็นค่า default `same_direction_loops` ที่อ้อมนอกกรอบทุกครั้ง)
- **deadhead แบบ rounded** : ย้ายระหว่าง pass โดย **อยู่ในกรอบพื้นที่** ไม่อ้อมออกนอก — ไหลลงตามแนว rail ฝั่งใกล้ เชื่อมปลายเลนทั้งสองด้วยส่วนโค้งรัศมี `turn_radius` พอดี (ปลายเลนถูกรันเข้าด้วย `turn_radius` ใน `_lane_x_bounds` ส่วนโค้งจึงไม่โป่งออกนอกกรอบ) เทียบกับ `outside` (อ้อมนอกกรอบ ต้องปิด keepout) และ `direct` (หมุนอยู่กับที่ด้วย Spin)
- **yaw-cut (enable_turn_yaw_cut:=true)**: ระหว่างเลี้ยว arc/deadhead node ฝ้าดู yaw จาก `/odometry/fusion_bno` เมื่อ heading เข้าใกล้ทิศของเลนถัดไป ($\text{error} < 0.25 \text{ rad} \approx 14^\circ$ หรือเลยเป้า) จะ **ยกเลิก goal ส่วนโค้งทันที** แล้วเริ่มเลนถัดไป — แก้ปัญหา arc ที่หมุนไม่ทัน (under-rotate) บนพื้นลื่น ไม่ให้หุ่นวนเลยเลน
- **gentle merge (เฉียงเข้าเลน)** ← ไม่ใช่ param ทำอัตโนมัติเสมอ (ใน `_run_lane`) : เมื่อเริ่มเลน

ใหม่

node อ่าน **ตำแหน่งจริง** ของหุ่นหลังเลี้ยว (จาก `/odometry/fusion_bno`) แล้วสร้างเลนเป็น **เส้นทแยงมุมตื้นๆ** จากจุดที่หุ่นอยู่จริง บรรจบเข้าหาเส้น grid ที่ปลายเลน แทนที่จะวาดเส้นนอน ที่ y เป้าทันที (ซึ่งจะกลายเป็น cross-track step ใหญ่ทำให้ RPP หมุนแก้แรงจนวน) → หุ่น "วิ่งตรงแล้วค่อยเฉียงเข้าเลน". **ไม่มี flag เปิด/ปิด** — โค้ดทำให้เองทุกครั้งที่ y จริง $\neq$ y เป้า (ส่วนการ "หมุนตั้งหัวก่อนเข้าเลน" แบบ Spin คนละตัว คือ `prealign_at_lane_entry` ซึ่ง **ปิดไว้**)

2) สั่ง Nav2 อย่างไร (Path Following) ● ใช้หลัก

ไฟล์ config: `config/nav2_params_tile.yaml`

Nav2 ที่ใช้คือ **controller_server** ปลั๊กอิน **RegulatedPurePursuitController (RPP)**

ทฤษฎี: **Pure Pursuit** — เปรียบเหมือน "วิ่งไล่ตามแครอทที่ห้อยอยู่ข้างหน้า"

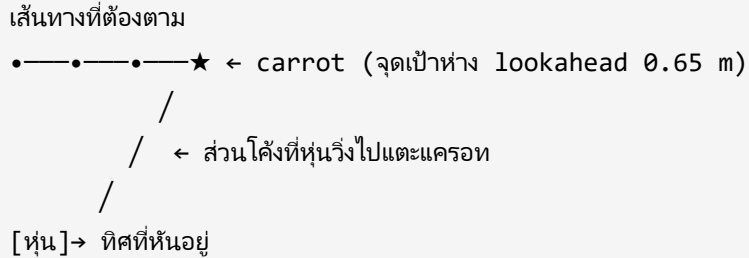
- หุ่นมองไปข้างหน้าบนเส้นทางเป็นระยะ `lookahead_dist` (0.65 m) เรียกจุดนั้นว่า "carrot" (แครอท)
- หุ่นพยายามวิ่งโค้งไปแตะแครอทนั้น — พอเข้าใกล้ แครอทก็เลื่อนหนีไปข้างหน้าเรื่อยๆ → หุ่นจึงวิ่งตามเส้น
- คำนวณส่วนโค้งจากตัวหุ่นไปแตะ carrot → ได้ความโค้ง (curvature)
- **เส้นโค้งมาจากสมการอะไร?** สมมติ carrot อยู่ห่าง `L_d` (lookahead) และเยื้องข้าง `y` (ในกรอบหุ่น) ส่วนโค้งวงกลมเดียวที่ลากจากหุ่นไปแตะ carrot ได้พอดี มีความโค้ง:

$$\begin{aligned} \kappa \text{ (curvature)} &= 2 \cdot y / L_d^2 && \leftarrow y = \text{ระยะเยื้องข้างของ carrot, } L_d = \text{lookahead_dist} \\ R \text{ (รัศมีโค้ง)} &= 1 / \kappa = L_d^2 / (2 \cdot y) \\ \omega \text{ (อัตราหมุน)} &= v \cdot \kappa && \leftarrow \text{แล้วส่งเป็น angular.z ใน /cmd_vel} \end{aligned}$$

carrot เยื้องมาก (y มาก) → κ มาก → โค้งแคบ; carrot อยู่ตรงหน้า ($y \approx 0$) → $\kappa \approx 0$ → วิ่งตรง

- "Regulated" = ลดความเร็วลงเมื่อเลี้ยวแคบ/ใกล้สิ่งกีดขวาง (ที่นี้ปรับให้ไม่ช้าเกินไป เพราะบนกระเบื้องล้อหลังกลมยิ่งช้ายิ่งสิ้น)

ภาพ Pure Pursuit:



โค้งมาก/โค้งน้อยปรับได้ไหม? ได้ — ปรับผ่าน `lookahead_dist` เป็นหลัก: ค่ามาก = หุ่นเล็งจุดไกล → โค้งกว้าง นุ่ม แต่ "ตัดโค้ง" (เข้าโค้งช้า); ค่าน้อย = เล็งใกล้ → โค้งแคบ เกาะเส้นแน่น แต่ส่าย นอกจากนี้ "Regulated scaling" ลดความเร็วเมื่อ R เล็กกว่า `regulated_linear_scaling_min_radius`

ค่าหลัก (จาก `nav2_params_tile.yaml` — ที่คำสั่งนี้ใช้):

- `desired_linear_vel: 0.20` m/s — ความเร็วเดินตรง
- `lookahead_dist: 0.65` m (ปรับช่วง `min 0.45` — `max 0.90`) — ระยะมองล่วงหน้า
- `regulated_linear_scaling_min_radius: 0.30` m, `..._min_speed: 0.20` — เลี้ยวแคบกว่านี้จึงเริ่มลดเร็ว
- `use_rotate_to_heading: false` — ปิดการหมุนตั้งหัวระดับ RPP (มันไปกวนโค้ง in-pass — ทำที่ระดับ node แทน)
- `goal_checker` : ถึงเป้าหมายอยู่ในรัศมี `xy_goal_tolerance 0.30` m และ `heading ± 0.50` rad

โค้งจริงในคำสั่งนี้ใช้เท่าไหน? เล่นตั้งใจให้เลี้ยวที่ $R \approx 0.6$ m (จาก `turn_radius:=0.60`).

ที่ $v=0.20$ m/s → ต้องใช้ $\omega = v/R = 0.20/0.6 \approx 0.33$ rad/s ซึ่งต่ำกว่าเพดาน mixer `max_w 0.45` พอดี (ออกแบบมาให้ไม่ชนเพดาน). ส่วนการ "ตัดโค้งทิ้งเมื่อหันทันแล้ว" ทำโดย yaw-cut ของ node (ข้อ 1.1)

Nav2 ออก `/cmd_vel` ที่ความเร็ว `controller_frequency: 20` Hz

Nav2 ไม่รู้จักล้อ มันสั่งแค่ "ตัวรถ" ว่าจะให้เดินเร็วแค่ไหน หมุนเร็วแค่ไหน (เหมือนสั่งพวงมาลัย+คันเร่ง ไม่ได้สั่งล้อแต่ละล้อ)

3) Nav2 → Mixer: แปลงความเร็วตัวรถเป็นความเร็ว 4 ล้อ ใช้หลัก

ไฟล์: `beach_wheel_mixer/src/wheel_mps_mixer.cpp` (C++), config: `mixer_turn06.yaml`

รับ `/cmd_vel` ($v = \text{linear.x}$, $\omega = \text{angular.z}$) → ออก `/wheel_cmd` = ความเร็ว 4 ล้อ (m/s)

หน้าที่เดียว: แปล "อยากให้รถเดิน+หมุนเท่านี้" → "ล้อแต่ละล้อต้องหมุนเร็วเท่าไร"

ทฤษฎี: Differential / Skid-steer kinematics

หุ่นแบบนี้แก้ด้วยการให้ **ล้อซ้าย-ขวาวิ่งคนละความเร็ว** (ไม่มีพวงมาลัย — เหมือนรถถัง/รถตักดิน)

สมการพื้นฐาน:

อยากเลี้ยวซ้าย → ล้อซ้ายช้าลง, ล้อขวาเร็วขึ้น

$$v_{\text{ซ้าย}} = v - \omega \cdot (\text{track}/2)$$

$$v_{\text{ขวา}} = v + \omega \cdot (\text{track}/2)$$

▲ `track` = ระยะห่างล้อซ้าย-ขวา

แต่หุ่นตัวนี้ **track** หน้ากับหลังไม่เท่ากัน (หน้าแคบ 0.734 m, หลังกว้าง 1.179 m)

จึงต้องแยกคำนวณต่อเพล:

$$FL = v - \omega_f \cdot (\text{front_track}/2)$$

$$FR = v + \omega_f \cdot (\text{front_track}/2) \quad \leftarrow \text{เพลหน้า (แคบ)}$$

$$RL = v - \omega_r \cdot (\text{rear_track}/2)$$

$$RR = v + \omega_r \cdot (\text{rear_track}/2) \quad \leftarrow \text{เพลหลัง (กว้าง)}$$

ค่าพารามิเตอร์ที่คำสั่งนี้ใช้ (จาก `mixer_turn06.yaml`):

param	ค่า	ความหมายสั้นๆ
<code>front_track_width</code>	0.734 m	ระยะล้อซ้าย-ขวา เพลหน้า
<code>rear_track_width</code>	1.179 m	ระยะล้อซ้าย-ขวา เพลหลัง
<code>wheelbase</code>	0.950 m	ระยะเพลหน้า-หลัง
<code>turn_gain_front</code>	1.5	เกนหมุนเพลหน้า (>1 = หมุนแรงขึ้น ขดเชยลื่น)
<code>turn_gain_rear</code>	1.45	เกนหมุนเพลหลัง

param	ค่า	ความหมายสั้นๆ
min_turn_radius	0.50 m	เลี้ยวแคบสุดที่ยอม (กันล้อในถอยหลัง)
rear_yaw_relief	0.15	ความแรงที่ "ผ่อนการหมุนเพลาลัง" ตอนเลี้ยวแคบ
max_v / max_w	0.35 / 0.45	เพดานความเร็วเดิน/หมุนของตัวรถ
max_wheel*_mps	FL 1.25, FR 1.10, RL 9.70, RR 8.60	เพดานความเร็วต่อล้อ
max_accel / max_decel	0.6 / 4.0 m/s ²	อัตราเร่ง/หน่วง (หน่วงแรงกว่า = หยุดไว)
cmd_timeout_ms	300	ไม่มี cmd ใหม่เกินนี้ → 0 ทุกล้อ
publish_rate_hz	50	ความถี่ออก /wheel_cmd

สิ่งที่ mixer ทำเพิ่มเพราะหุ่นไม่สมมาตร + ลื่นบนพื้นแข็ง:

- **turn_gain_front/rear (1.5/1.45):** คูณเข้ากับ ω แยกหน้า-หลัง ($w_f = \omega \cdot \text{gain}_f$, $w_r = \omega \cdot \text{gain}_r$)
ชดเชยการลื่น/under-rotate
- **min_turn_radius (0.50 m):** จำกัด ω ไม่ให้เกิน $|v|/R_{\min}$ (เลี้ยวแคบเกินจนล้อในต้องถอยหลัง — `limit_w_for_turn_radius`)
- **rear_yaw_relief (0.15) — อธิบายละเอียด:** เพลาลังเป็นล้อกลมกว้าง (1.179 m) ตอนเลี้ยวแคบมันจะ "ครูด/ฝืน" ตัวรถ โค้ดจึง ลดการหมุนของเพลาลังลงเมื่อโค้งแคบ ด้วยตัวคูณ:

```

conflict = curvature * wheelbase = (|w|/v) * 0.950
relief_factor = 1 / (1 + rear_yaw_relief * conflict2)      ← rear_yaw_relief = 0.15
ω_หลังจริง = ω * turn_gain_rear * relief_factor

```

ยิ่งเลี้ยวแคบ (curvature สูง) → conflict สูง → relief_factor เข้าใกล้ 0 → เพลาลังหมุนน้อยลง
ปล่อยให้ "ตามหน้า" แทนที่จะฝืนครูดพื้น. ตอนเลี้ยวกว้าง/วิ่งตรง relief_factor ≈ 1 (ไม่ผ่อน)

- **in-place turn:** ถ้า $|v| \leq 0.03$ แต่ $|w| \geq 0.03$ → หมอดหมุนอยู่กับที่ (ใช้เกนคนละชุด `turn_gain*_in_place`, ไม่ใช่ `rear_yaw_relief`)
- **common scale — อธิบายละเอียด:** หลังคำนวณความเร็ว 4 ล้อแล้ว ถ้าล้อใดเกินเพดานของมัน (เช่น $FL > 1.25$) โค้ดจะหา ตัวคูณเดียว `scale = min(เพดาน_i / |ความเร็ว_i|)` แล้ว คูณกลับทั้ง 4 ล้อ

พร้อมกัน:

ถ้า FL=1.50 แต่เพดาน 1.25 → $scale = 1.25/1.50 = 0.83$ → คูณทุกล้อด้วย 0.83

ทำไมต้องคูณทั้ง 4? เพราะ อัตราส่วนความเร็วระหว่างล้อ = ความโค้งของการเลี้ยว ถ้าหันแค่ล้อเดียว อัตราส่วนเพี้ยน หุ่นจะเลี้ยวผิดมุม → คูณพร้อมกันทำให้ "ช้าลงทั้งคันแต่ยังเลี้ยวโค้งเดิม"

- **ramp (accel/decel):** จำกัดอัตราเร่ง 0.6 / หน่วง 4.0 m/s² ให้ออกตัวนุ่ม หยุดไว
- **watchdog:** ถ้าไม่มี /cmd_vel ใหม่ภายใน 300 ms → สั่ง 0 ทุกล้อ
- เผยแพร่ /wheel_cmd ที่ 50 Hz

4) Mixer → ESP32 Bridge: สะพาน ROS2 ↔ Serial

● ใช้หลัก

ไฟล์: `beach_robot_esp32_bridge/esp32_bridge_node.py` (Python)

bridge เป็นตัวกลางระหว่างโลก ROS2 กับ ESP32 ผ่านพอร์ต Serial

`/dev/ttyESP32` @ **230400 baud** ด้วยโปรโตคอล **JSON** ที่ละบรรทัด

(เหมือน "ล่าม" แปลภาษา ROS2 ↔ ภาษา ESP32 ไป-กลับ)

baud กับ Hz ต่างกันยังไง? (สับสนกันบ่อย)

- **baud (230400)** = ความเร็ว "ตัวอักษร" ริงบนสาย = 230,400 บิต/วินาที → เป็น **ขนาดท่อ** (ท่อกว้างแค่ไหน) ส่งได้เร็วสุดเท่านี้ แต่ไม่ได้แปลว่าเราส่งเต็มท่อตลอด
 - **Hz (20)** = เรา "ยัดแพ็กเก็ตคำสั่งล้อ" เข้าท่อ **กี่ครั้งต่อวินาที** = **ความถี่การส่ง**
 - เปรียบ: baud = ถนนกว้างกี่เลน, Hz = เราปล่อยรถออกกี่คันต่อวินาที (1 แพ็กเก็ต JSON ~50–80 ไบต์)
- ใช้ท่อแค่เลี้ยวเดียว ท่อ 230400 จึงเหลือเพื่อสำหรับ 20 Hz)

สรุปอัตราจริงในระบบนี้:

- **ROS2 → ESP32 (คำสั่งล้อ): 20 Hz** ← `wheel_cmd_send_rate_hz:=20` ที่ launch ตั้งหับ (ค่า default ในโค้ด node คือ 30 Hz แต่ **คำสั่งนี้ใช้ 20 Hz**)
- **ESP32 → ROS2 (telemetry): ~10 Hz** (ทุก 100 ms)
- หมายเหตุ: mixer ผลิต `/wheel_cmd` ที่ **50 Hz** แต่ bridge เก็บคำสั่งล่าสุดไว้แล้ว **ส่งออกท่อ serial แค่ 20 Hz** (ทุกครั้งที่ timer เต็ม หยิบคำสั่งล่าสุดไปส่ง) — เลขสามตัวนี้ไม่ขัดกัน

ขาออก (ROS2 → ESP32):

- รับ `/wheel_cmd` แล้วเก็บไว้ ส่งออกเป็นจังหวะคงที่ด้วย timer **20 Hz**
(ค่า `wheel_cmd_send_rate_hz` ที่ launch ตั้งไว้ปัจจุบัน; ค่า default ใน node เองคือ 30 Hz)
→ `{"wheel_cmd": [FL, FR, RL, RR], "cmd_seq": n}`
(`cmd_seq` เป็นเลขลำดับให้ ESP32 ตรวจสอบว่าแพ็กเก็ตหายไหม)
- **Stale timeout 0.5 s:** ถ้าไม่ได้ค่าใหม่ → ส่ง `[0,0,0,0]` (กันรถวิ่งต่อเมื่อ Nav2 ค้าง)
- **E-STOP:** พัง `/e_stop` (มือกด) + `/safety/e_stop` (ZED ตรวจสอบสิ่งกีดขวาง) แบบ OR
ถ้า active → **บังคับส่ง 0 ทุกสั่** ทับคำสั่งทุกอย่าง
- ส่งคำสั่งอื่น: `buzzer_duration`, `vibration_enable` (มอเตอร์สั่นรอนทราย 0), `json_cmd` (จูน PID 0)

ขาเข้า (ESP32 → ROS2): อ่าน Serial ใน thread แยก แดก JSON เป็น topic:

- `enc_vel` → `/enc_vel` (ความเร็ว 4 ล้อจาก encoder)
- `imu_quat/imu_gyro/imu_lin_acc` → `/imu/data` (`sensor_msgs/Imu`)
- `ultra [L,M,R]` → `/ultrasonic/{left,middle,right}` (`sensor_msgs/Range`)

ตัวกรอง encoder: หักตัวอย่างที่ล้อไต่ > 3.0 m/s หรือกระโดด > 1.0 m/s/step
(กัน noise/spike จาก encoder ทำ odometry เพี้ยน)

5) ESP32 ควบคุมล้ออย่างไร 🌀 ใช้หลัก

ไฟล์: `firmware/src/main_esp.cpp` (อ้างอิงเท่านั้น แพลชด้วย PlatformIO แยก repo)

ESP32 รับ `wheel_cmd` (ความเร็วเป้าหมาย m/s ของแต่ละล้อ) แล้วคุมมอเตอร์ให้วิ่งตามนั้น

ทฤษฎี: PID closed-loop control + feed-forward

นึกถึง **ระบบ cruise control ของรถ** — ตั้งความเร็วไว้ พอขึ้นเนินรถช้าลง ระบบก็เหยียบคันเร่งเพิ่มเอง
ให้กลับมาเร็วเท่าเดิม PID ทำแบบเดียวกันกับล้อแต่ละล้อ ทุกรอบ loop:

1. **Feed-forward (เดาก่อน):** เดาค่าสั่งเบื้องต้น `u_ff = target_mps / V_MAX` (สเกลเป็น -1..+1)
— เหมือนกะน้ำหนักเท้าคร่าวๆ ก่อน ไม่ต้องรอ error
2. **อ่าน encoder จริง** (ผ่าน low-pass filter, `enc_vel_corr`) — ดูว่าล้อหมุนจริงเร็วเท่าไร
3. **PID (แก้ตามจริง):** `u_pid = Kp·error + Ki·ferror dt + Kd·d(error)/dt`
โดย `error = (ความเร็วเป้าหมาย - ความเร็วที่วัดได้)`

- **Kp** = แก้ตามความต่างตอนนี้, **Ki** = สะสมแก้ความต่างที่ค้างนานๆ (เช่นแรงต้านคงที่ตอนขึ้นเนิน)
- $Kp = [0.15, 0.18, 0.04, 0.03]$, $Ki = [0.08, 0.08, 0.05, 0.05]$, $Kd = 0$
- หน้ากับหลังเกนต่างกันเพราะมอเตอร์/หดรอบคนละแบบ
- มี **anti-windup**: ถ้าคำสั่งชนเพดาน (อิมตัว) แล้วยังอยากเพิ่มทางเดิม จะ **ไม่สะสม Ki** ต่อ (กันค่าสะสมพุ่งจนแก้ยากตอนพื้นเนิน)

4. **รวม:** `u_out = clamp(u_ff + u_pid, -1, +1)`
5. **drive floor:** ถ้าความเร็วต่ำมาก เติม PWM ชั้นต่ำเพื่อเอาชนะแรงเสียดทาน (กันอาการลื้อค้าง/กระตุกที่ความเร็วต่ำ — stick-slip)
6. แปลง `u_out` → **PWM** ส่งเข้า motor driver → ล้อหมุน (`set_motor_speeds`)

ความปลอดภัย:

- ถ้า `|target| < deadband` → **สั่ง 0 + reset PID** — อธิบายศัพท์:
 - **target (cmd_target)** = ความเร็ว เป้าหมาย ของล้อนั้น (m/s) ที่รับมาจาก `wheel_cmd`
 - **deadband (V_CMD_DEADBAND)** = **เกณฑ์ขั้นต่ำ = 0.005 m/s** (5 มิลลิเมตร/วินาที)
 - แปลว่า: ถ้าสั่งให้ลื้อวิ่งช้ากว่า 5 mm/s (แทบเท่ากับ "ให้หยุด") → ฟันธงเป็น **0** ทันที ทั้ง feed-forward และ PID, แล้ว `pid.reset()` ล้างค่าสะสม (integral) ทิ้ง
 - **ทำไมต้อง reset PID?** ถ้าไม่ล้าง ค่า integral ที่ค้างอยู่จะดันให้ลื้อ "คืบ" ทั้งที่สั่งให้หยุด → reset กันหุนไหลตอนควรนิ่ง
- จำกัดความเร็วต่อล้อ (`V_MAX` ต่างกันมาก: หน้า ~1.2, หลัง ~9 m/s เพราะอัตราทดต่างกัน)
- ESP32 อ่าน IMU (BNO055), ultrasonic, encoder เอง

สรุปวงรอบล่าง: Nav2/mixer สั่งแค่ "อยากให้ลื้อนี้วิ่ง X m/s" ส่วนการ "ทำให้ได้ X จริง" เป็นหน้าที่ PID บน ESP32 ที่เทียบกับ encoder — ทำให้หุ่นทนต่อ ภาระที่เปลี่ยน (ขึ้นเนินทราย, แรงต้านต่างกัน) ได้เอง

6) ข้อมูลที่ ESP32 ส่งกลับ (Telemetry) ● ใช้หลัก

ESP32 ส่ง JSON กลับทุก `TELEMETRY_PERIOD_MS = 100 ms` (10 Hz):

```
{
  "enc_vel":      [v_FL, v_FR, v_RL, v_RR],    // ความเร็วล้อจาก encoder (m/s)
  "imu_quat":     [x, y, z, w],                // ทิศทางหุ่น (quaternion จาก BNO055)
  "imu_gyro":     [gx, gy, gz],                // อัตราหมุน (rad/s)
  "imu_lin_acc":  [ax, ay, az],                // ความเร่งเชิงเส้น (m/s²)
  "ultra":        [left, middle, right]        // ระยะสิ่งกีดขวาง (m)
}
```

ค่าเหล่านี้คือ "ประสาทสัมผัส" ที่ทำให้ ROS2 รู้ว่าเกิดอะไรขึ้นจริง (ล้อหมุนจริงไหม หันไปทางไหน มีอะไรขวางหน้าหรือเปล่า)

7) ปิดวงรอบ: Odometry + EKF (หุ่นรู้ตำแหน่งตัวเอง) ●

ใช้หลัก

ข้อมูลจาก ESP32 ถูกนำไปคำนวณตำแหน่ง เพื่อป้อนกลับให้ Nav2

ก) Wheel Odometry (wheel_odometry_node.py) ●

แปลง `/enc_vel` (4 ล้อ) → ตำแหน่ง `/wheel/odom` ด้วยจลนศาสตร์ (forward kinematics)

— หลักการเหมือน "นับรอบล้อแล้วเดาว่าเดินมาไกลแค่ไหน" (dead reckoning):

```
vx = ¼ • (FL+FR+RL+RR)                // ความเร็วเดินหน้า = เฉลี่ย 4 ล้อ
wz = (front_track • (FR-FL) + rear_track • (RR-RL)) / (front_track² + rear_track²)
vx_จริง = vx • linear_scale             // linear_scale = 1.625 (กระเบื้อง)
```

แล้ว integrate (บวกสะสมทีละ step): `yaw += wz • dt`, `x += vx • cos(yaw) • dt`, `y += vx • sin(yaw) • dt`

- **linear_scale** ชดเชยการลื่นของล้อ: encoder บอกระยะหนึ่ง แต่จริงวิ่งได้ไกลกว่า (กระเบื้อง 1.625, หวาย 1.19 — จูนด้วยตลับเมตร)
- **angular_scale = 1.0 (ไม่แตะ)**: yaw จาก wheel-odom ลื่นและซ้ำ/ขวาไม่เท่ากัน สเกลเดียวแก้ไม่ได้ → ให้ **BNO055 เป็นเจ้าของ heading** แทน

ข) EKF (Extended Kalman Filter) — `robot_localization/ekf_node`, config

`config/ekf_imu_compare.yaml` ●

รวม (fuse) ข้อมูลหลายแหล่งที่มี noise ต่างกัน ให้ได้ตำแหน่งที่นิ่งและแม่นยำกว่าทุกแหล่งเดียว

— เปรียบเหมือน เอา **GPS (บอกตำแหน่ง) มารวมกับเข็มทิศ (บอกทิศ)** แล้วเชื่อแต่ละอันในเรื่องที่มันเก่ง

โหนด EKF ที่ออก `/odometry/fusion_bno` (ชื่อ `ekf_wheel_bno` , `two_d_mode` , 30 Hz) เลือกรับเฉพาะตัวแปรที่แต่ละ sensor ทำได้ดีที่สุด:

- **ความเร็วเดินทาง vx** จาก **wheel odom** (`/wheel/odom`) — ดีระยะสั้น แต่ yaw จากล้อสั่น/สะสม error จึง **ไม่เอา yaw จากล้อ**
- **heading yaw** สัมบูรณ์ จาก **IMU BNO055** (`/imu/data`) — ดีเรื่องมุม ไม่ดริฟต์เร็ว → BNO เป็นเจ้าของ heading
- ○ (ถ้าเปิด `use_gnss:=true`) ตำแหน่งสัมบูรณ์จาก **GNSS RTK** ผ่าน `navsat_transform` → `/odometry/gps` (คำสั่งทดสอบบนกระเบื้องใช้ `use_gnss:=false` จึง **ไม่ใช้ GNSS** — เป็นของเสริม)

EKF ออก `/odometry/fusion_bno` ← นี่คือตำแหน่งที่ **Nav2 ใช้จริง** (`bt_navigator` ตั้ง `odom_topic: /odometry/fusion_bno` ; วนกลับไปข้อ 2)

ทฤษฎี Kalman อย่างย่อ: ทำนายตำแหน่งถัดไปจากโมเดลการเคลื่อนที่ แล้ว "แก้" ด้วยค่าวัดจริง โดยถ่วงน้ำหนักตามความน่าเชื่อถือ (covariance) ของแต่ละ sensor → sensor ไหนแม่นยำ ก็ถูกถ่วงน้ำหนักน้อยลงอัตโนมัติ (เหมือนเราเชื่อเพื่อนที่แม่นยำมากกว่า)

8) ความปลอดภัยและการหยุด (ทุกชั้นมี watchdog)

หลักคิด: **"กันไว้หลายชั้น" (defense in depth)** — ชั้นไหนพังหรือค้าง ชั้นล่างยังหยุดหุ่นได้

ชั้น	กลไกหยุด	จำเป็น
Nav2	ZED ตรวจสอบสิ่งกีดขวาง → costmap → หยุด/วางแผนใหม่	○ (ZED ปิดได้ด้วย <code>use_zed:=false</code>)
Coverage node	FrontBoxMonitor (ZED) → override <code>/cmd_vel = 0</code> + บัชเซอร์	○ (ฟัง ZED)
esp32_bridge	E-STOP (มือ/safety) → บังคับ <code>wheel_cmd = 0</code> ; stale 0.5 s → 0	●
mixer	ไม่มี cmd ใหม่ 300 ms → 0 ทุกล้อ	●
ESP32	target < deadband → 0; จำกัด <code>V_MAX</code> ต่อล้อ	●

- E-STOP มือกด + watchdog ของ mixer/ESP32 ทำงานเสมอ ไม่ขึ้นกับ ZED/GNSS
- การหยุดเพราะ "เห็นสิ่งกีดขวาง" ฟังก์ชัน ZED ถ้าปิด ZED จะเหลือแค่ ultrasonic ช่วยเตือน

📁 สรุปสั้นๆ ใครทำอะไร (+ จำเป็นแค่ไหน)

ส่วน	ภาษา	input	output	หน้าที่	จำเป็น
Coverage planner	Python	กรอบพื้นที่	path (ส่ง goal เป็นช่วง)	วางแผนไถนา + คำนวณครอบคลุม	●
Nav2 RPP	(Nav2)	path + ตำแหน่ง	<code>/cmd_vel</code> (v, ω)	ติดตามเส้นทาง	●
mixer	C++	<code>/cmd_vel</code>	<code>/wheel_cmd</code> (4 ล้อ)	จลนศาสตร์ skid-steer	●
esp32_bridge	Python	<code>/wheel_cmd</code>	Serial JSON	สะพาน ROS2↔ Serial + safety	●
ESP32	C++	wheel_cmd	PWM + telemetry	PID คมล้อ + อ่านเซนเซอร์	●
wheel_odom + EKF	Python	enc/imu	<code>/odometry/fusion_bno</code>	บอกหุ่นว่าอยู่ตรงไหน (ปิดวงรอบ)	●
GNSS / navsat	Python	<code>/gps/fix</code>	<code>/odometry/gps</code>	ตำแหน่งสัมบูรณ์กลางแจ้ง	○
ZED costmap / safety	C++	point cloud	costmap + <code>/safety/e_stop</code>	ตรวจสอบสิ่งกีดขวาง	○

ส่วน	ภาษา	input	output	หน้าที่	จำเป็น
keepout mask	(Nav2)	mask map	แนวห้ามเข้า	กั้นขอบเขต พื้นที่	○

สายหลักที่ขาดไม่ได้ (●): planner → Nav2 → mixer → bridge → ESP32 → เซนเซอร์ → odometry/EKF → กลับเข้า Nav2

ทำซ้ำหลายสิบครั้งต่อวินาที จนหุ่นปุพรมเก็บขยะครบทั้งพื้นที่

ของเสริม (○) เปิดเมื่อต้องการ: GNSS (กลางแจ้ง RTK), ZED (เห็นสิ่งกีดขวาง), keepout (กั้นเขต) —
คำสั่งทดสอบบนกระเบื้องปิดทั้งหมด ใช้แค่สายหลักก็วิ่งครบรูปได้